

FiPro: Personal Finance Management Application - Design Document v2.0

Document Information

Version: 2.0

Date: August 24, 2025

Author: Software Development Team

Project Name: FiPro (Financial Project)

Status: Design Phase

1. Executive Summary

1.1 Project Overview

FiPro is a comprehensive personal finance management application designed to automate the tedious process of tracking expenses and income through intelligent PDF bank statement parsing. The application transforms unstructured financial data from multiple banking institutions into actionable insights, eliminating manual data entry and providing users with a unified view of their financial activity.

1.2 Business Objectives

- **Automate Data Entry:** Extract transaction data from PDF bank statements across multiple financial institutions
- **Unify Financial Data:** Consolidate transactions from various accounts into a standardized format
- **Provide Actionable Insights:** Enable transaction categorization and spending pattern visualization
- **Ensure Data Accuracy:** Maintain high precision in data extraction and processing
- **Enable Extensibility:** Design modular system architecture for easy addition of new banks and features

1.3 Target Users

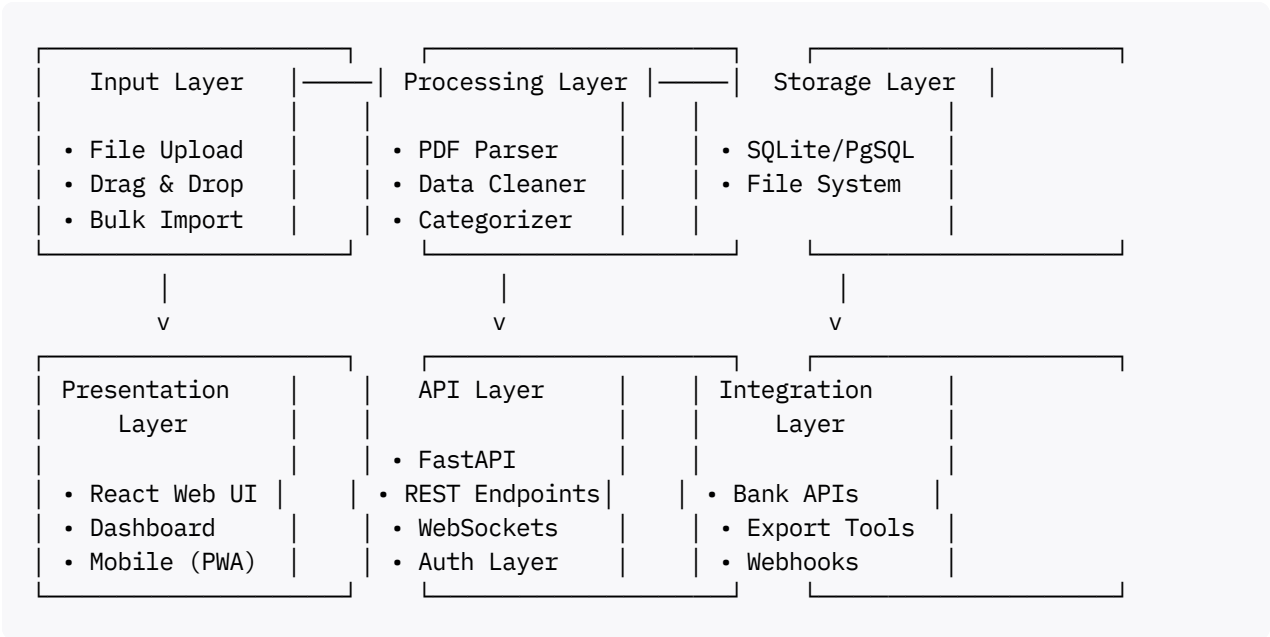
- **Primary:** Tech-savvy individuals managing multiple bank accounts
- **Secondary:** Small business owners tracking business expenses
- **Tertiary:** Financial advisors managing client portfolios

2. System Architecture

2.1 Architecture Philosophy

The system employs a **microservices-inspired modular architecture** built around a data processing pipeline. Each component is designed as a distinct module performing specific tasks while maintaining loose coupling and high cohesion.

2.2 High-Level Architecture



2.3 Component Breakdown

2.3.1 File Processing Pipeline

File Orchestrator

- **Purpose:** Master controller for file lifecycle management
- **Responsibilities:**
 - Monitor input directories for new files
 - Identify bank type from filename patterns or content analysis
 - Route files to appropriate parsers
 - Manage file states (processing, completed, failed)
 - Implement retry mechanisms with exponential backoff

PDF Parser Engine

- **Purpose:** Extract raw text content from PDF statements
- **Strategy Pattern Implementation:**
 - Individual parsers for each supported bank (HDFC, SBI, Axis, etc.)

- Configurable parsing rules through YAML configurations
- Support for password-protected PDFs
- OCR capabilities for scanned statements using Tesseract

Transaction Extractor

- **Purpose:** Convert raw text into structured transaction records
- **Features:**
 - Regex-based pattern matching for different statement formats
 - Context-aware parsing (handling multi-line transactions)
 - Date normalization across different formats
 - Amount parsing with currency symbol handling

Data Cleaner & Standardizer

- **Purpose:** Normalize and validate extracted data
- **Operations:**
 - Remove currency symbols and formatting
 - Standardize date formats (ISO 8601)
 - Categorize transactions using rule-based engine
 - Detect and flag potential duplicates

2.3.2 API Backend

FastAPI Server

- **Authentication:** JWT-based with refresh tokens
- **Rate Limiting:** Configurable per-endpoint limits
- **Documentation:** Auto-generated OpenAPI/Swagger docs
- **Monitoring:** Request logging and performance metrics

Core Endpoints:

GET	/api/v1/transactions	# List transactions with filtering
POST	/api/v1/transactions	# Create manual transaction
PUT	/api/v1/transactions/{id}	# Update transaction
DELETE	/api/v1/transactions/{id}	# Delete transaction
POST	/api/v1/upload	# Upload bank statement
GET	/api/v1/categories	# List available categories
POST	/api/v1/categories	# Create custom category
GET	/api/v1/analytics/summary	# Financial summary
GET	/api/v1/analytics/trends	# Spending trends

2.3.3 Frontend Application

Technology Stack:

- **Framework:** React 18 with TypeScript
- **State Management:** Redux Toolkit + RTK Query
- **UI Library:** Material-UI (MUI) v5
- **Styling:** Emotion (CSS-in-JS)
- **Charts:** Recharts for data visualization
- **Build Tool:** Vite for fast development

Key Features:

- **Dashboard:** Real-time financial overview with key metrics
- **Transaction Table:** Sortable, filterable transaction list with bulk operations
- **Upload Interface:** Drag-and-drop file upload with progress tracking
- **Analytics:** Interactive charts showing spending patterns and trends
- **Categorization:** Manual and rule-based transaction categorization
- **Export:** CSV/Excel export functionality

3. Data Model & Schema

3.1 Core Transaction Schema

```
CREATE TABLE transactions (  
    id SERIAL PRIMARY KEY,  
    transaction_date DATE NOT NULL,  
    description TEXT NOT NULL,  
    amount DECIMAL(12,2) NOT NULL,  
    transaction_type VARCHAR(10) CHECK (transaction_type IN ('debit', 'credit')),  
    category_id INTEGER REFERENCES categories(id),  
    source_bank VARCHAR(50) NOT NULL,  
    source_file VARCHAR(255) NOT NULL,  
    raw_data JSONB,  
    hash VARCHAR(64) UNIQUE NOT NULL,  
    balance DECIMAL(12,2),  
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
    updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
);  
  
CREATE INDEX idx_transactions_date ON transactions(transaction_date);  
CREATE INDEX idx_transactions_category ON transactions(category_id);  
CREATE INDEX idx_transactions_bank ON transactions(source_bank);  
CREATE INDEX idx_transactions_hash ON transactions(hash);
```

3.2 Categories Schema

```
CREATE TABLE categories (  
  id SERIAL PRIMARY KEY,  
  name VARCHAR(100) UNIQUE NOT NULL,  
  color VARCHAR(7) DEFAULT '#6366f1',  
  icon VARCHAR(50) DEFAULT 'category',  
  is_system BOOLEAN DEFAULT FALSE,  
  parent_id INTEGER REFERENCES categories(id),  
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
);  
  
-- Pre-populate system categories  
INSERT INTO categories (name, is_system) VALUES  
( 'Shopping', TRUE),  
( 'Food & Dining', TRUE),  
( 'Transportation', TRUE),  
( 'Entertainment', TRUE),  
( 'Bills & Utilities', TRUE),  
( 'Healthcare', TRUE),  
( 'Income', TRUE),  
( 'Transfer', TRUE);
```

3.3 Bank Configuration Schema

```
CREATE TABLE bank_configs (  
  id SERIAL PRIMARY KEY,  
  bank_name VARCHAR(100) UNIQUE NOT NULL,  
  statement_patterns JSONB NOT NULL,  
  date_format VARCHAR(50) NOT NULL,  
  amount_regex VARCHAR(255) NOT NULL,  
  description_regex VARCHAR(255) NOT NULL,  
  is_active BOOLEAN DEFAULT TRUE,  
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
);
```

4. Security & Privacy Framework

4.1 Data Protection

- **Encryption at Rest:** AES-256 encryption for sensitive data
- **Encryption in Transit:** TLS 1.3 for all API communications
- **Local Storage:** SQLite with encrypted database files
- **File Security:** Automatic file cleanup after processing

4.2 Authentication & Authorization

- **JWT Tokens:** Short-lived access tokens (15 minutes) with refresh mechanism
- **Password Security:** bcrypt hashing with salt rounds
- **Session Management:** Secure session handling with automatic logout
- **API Security:** Rate limiting and request validation

4.3 Privacy Compliance

- **Data Minimization:** Only store essential transaction data
- **User Control:** Easy data export and deletion capabilities
- **Audit Logging:** Track all data access and modifications
- **GDPR Compliance:** Right to erasure and data portability

5. Error Handling & Logging

5.1 Error Classification

Critical Errors (Level 1):

- Database connection failures
- File system permissions issues
- Authentication/authorization failures

Processing Errors (Level 2):

- PDF parsing failures
- Invalid file formats
- Data validation errors

User Errors (Level 3):

- Invalid input data
- Missing required fields
- Duplicate uploads

5.2 Logging Strategy

```
# Logging Configuration
LOGGING_CONFIG = {
    'version': 1,
    'disable_existing_loggers': False,
    'formatters': {
        'detailed': {
            'format': '{levelname} {asctime} [{name}] {message}',
            'style': '{'
```

```

    }
    },
    'handlers': {
        'file': {
            'class': 'logging.handlers.RotatingFileHandler',
            'filename': 'logs/fipro.log',
            'maxBytes': 10485760, # 10MB
            'backupCount': 5,
            'formatter': 'detailed'
        }
    },
    'loggers': {
        'fipro': {
            'handlers': ['file'],
            'level': 'INFO',
            'propagate': False
        }
    }
}

```

5.3 File Lifecycle Management

```

Input Directory (data/input/)
  ↓ [Processing]
Processing Directory (data/processing/)
  ↓ [Success/Failure]
  └─ Success → Archive Directory (data/processed/)
  └─ Failure → Error Directory (data/failed/)

```

6. Development Roadmap

6.1 Phase 1: Core Infrastructure (Weeks 1-4)

Milestone 1: Foundation

- [x] Basic PDF text extraction scripts
- [x] SBI and HDFC statement parsers
- [] **Priority Tasks:**
 - [] Implement unified Transaction data model
 - [] Create robust file orchestrator with error handling
 - [] Set up SQLite database with SQLAlchemy ORM
 - [] Implement duplicate detection using hash-based approach
 - [] Create data cleaning and standardization pipeline

Deliverables:

- Working PDF processing pipeline

- SQLite database with core schema
- Basic transaction extraction for 2+ banks

6.2 Phase 2: API & Frontend Foundation (Weeks 5-8)

Milestone 2: Basic Application

- ☐ **Backend Development:**
 - ☐ FastAPI server with authentication
 - ☐ RESTful API endpoints for transactions
 - ☐ File upload handling with validation
 - ☐ Error handling and logging framework
- ☐ **Frontend Development:**
 - ☐ React application setup with TypeScript
 - ☐ Material-UI component library integration
 - ☐ Transaction table with sorting/filtering
 - ☐ File upload interface with progress tracking

Deliverables:

- Functional web application
- Complete CRUD operations for transactions
- File upload and processing workflow

6.3 Phase 3: Advanced Features (Weeks 9-12)

Milestone 3: Enhanced Functionality

- ☐ **Smart Categorization:**
 - ☐ Rule-based auto-categorization engine
 - ☐ Machine learning model for transaction classification
 - ☐ User-defined category rules
- ☐ **Analytics & Visualization:**
 - ☐ Interactive dashboard with key metrics
 - ☐ Spending trend analysis
 - ☐ Budget tracking and alerts
- ☐ **Additional Banks:**
 - ☐ Add support for 5+ major Indian banks
 - ☐ Credit card statement parsing
 - ☐ Investment account statements

Deliverables:

- Intelligent transaction categorization
- Comprehensive analytics dashboard
- Multi-bank support

6.4 Phase 4: Production Ready (Weeks 13-16)

Milestone 4: Deployment & Optimization

- **[] Production Setup:**
 - [] PostgreSQL migration for production
 - [] Docker containerization
 - [] CI/CD pipeline setup
 - [] Performance optimization
- **[] Security Hardening:**
 - [] Security audit and penetration testing
 - [] Data encryption implementation
 - [] Compliance documentation
- **[] User Experience:**
 - [] Mobile-responsive design
 - [] Progressive Web App (PWA) features
 - [] User onboarding flow

Deliverables:

- Production-ready application
- Security compliance certification
- Mobile-optimized interface

7. Technology Stack

7.1 Backend Stack

- **Language:** Python 3.11+
- **Web Framework:** FastAPI 0.104+
- **Database:** SQLite (development) → PostgreSQL 15+ (production)
- **ORM:** SQLAlchemy 2.0+ with async support
- **PDF Processing:** PyMuPDF (fitz) + pdfplumber
- **OCR:** Tesseract with pytesseract wrapper

- **Authentication:** JWT with python-jose
- **Task Queue:** Celery with Redis (future enhancement)

7.2 Frontend Stack

- **Framework:** React 18 with TypeScript
- **Build Tool:** Vite 4.0+
- **State Management:** Redux Toolkit + RTK Query
- **UI Framework:** Material-UI (MUI) v5
- **Charts:** Recharts for data visualization
- **HTTP Client:** Axios with interceptors
- **Form Handling:** React Hook Form with Zod validation

7.3 Infrastructure & DevOps

- **Containerization:** Docker + Docker Compose
- **Web Server:** Nginx (production)
- **Process Manager:** Gunicorn with Uvicorn workers
- **Monitoring:** Prometheus + Grafana (future)
- **Logging:** Structured logging with JSON format
- **Version Control:** Git with conventional commits

7.4 Development Tools

- **Code Quality:** Black, isort, flake8, mypy
- **Testing:** Pytest for backend, Jest + React Testing Library for frontend
- **API Documentation:** Auto-generated with FastAPI/OpenAPI
- **Database Migrations:** Alembic
- **Package Management:** Poetry (Python), npm/yarn (JavaScript)

8. Performance & Scalability

8.1 Performance Targets

- **File Processing:** < 30 seconds for typical bank statement (1-50 transactions)
- **API Response Time:** < 500ms for transaction queries
- **Database Queries:** < 100ms for filtered transaction lists
- **File Upload:** Support files up to 10MB
- **Concurrent Users:** 100+ simultaneous users (production target)

8.2 Scalability Considerations

- **Horizontal Scaling:** Stateless API design for easy load balancing
- **Database Optimization:** Proper indexing and query optimization
- **Caching Strategy:** Redis for session management and frequent queries
- **File Storage:** Scalable file storage with cloud providers
- **Background Processing:** Async task processing for heavy operations

8.3 Monitoring & Alerting

- **Application Metrics:** Response times, error rates, throughput
- **Infrastructure Metrics:** CPU, memory, disk usage
- **Business Metrics:** Processing success rates, user activity
- **Alerting:** Email/Slack notifications for critical issues

9. Testing Strategy

9.1 Testing Pyramid

- **Unit Tests:** Individual component testing (80% coverage target)
- **Integration Tests:** API endpoint and database interaction testing
- **End-to-End Tests:** Complete user workflow testing
- **Performance Tests:** Load testing with realistic data volumes

9.2 Test Data Management

- **Sample Statements:** Anonymized bank statements for each supported bank
- **Test Database:** Separate test database with known data sets
- **Mock Services:** Mock external services for consistent testing
- **Continuous Testing:** Automated test runs on every commit

10. Future Enhancements

10.1 Advanced Features (Post-MVP)

- **Budgeting Module:** Monthly budget setting and tracking with alerts
- **Investment Tracking:** Integration with brokerage APIs and portfolio management
- **Bill Reminders:** Automated bill detection and payment reminders
- **Financial Goals:** Savings goals with progress tracking
- **Multi-Currency Support:** Handle international transactions and currencies

10.2 Technical Enhancements

- **Machine Learning:** Advanced transaction categorization with ML models
- **Real-time Updates:** WebSocket connections for real-time data updates
- **Mobile Apps:** Native iOS/Android applications
- **API Integrations:** Direct bank API connections (Open Banking)
- **Cloud Deployment:** Kubernetes-based deployment on AWS/GCP

10.3 Business Features

- **Multi-User Support:** Family account management
- **Data Export/Import:** Comprehensive data portability
- **Reporting:** Advanced financial reports and tax preparation
- **Notifications:** Smart alerts and insights
- **Third-party Integrations:** Accounting software, tax tools

11. Risk Assessment & Mitigation

11.1 Technical Risks

Risk	Impact	Probability	Mitigation
PDF parsing accuracy	High	Medium	Implement fallback OCR, manual verification UI
Database performance	Medium	Low	Proper indexing, query optimization, caching
Security vulnerabilities	High	Low	Regular security audits, penetration testing
Third-party dependencies	Medium	Medium	Version pinning, dependency monitoring

11.2 Business Risks

Risk	Impact	Probability	Mitigation
Regulatory compliance	High	Low	Legal consultation, privacy by design
Competition	Medium	High	Focus on unique value proposition, rapid iteration
User adoption	High	Medium	User research, iterative design improvements
Data privacy concerns	High	Low	Transparent privacy policy, local-first approach

12. Success Metrics

12.1 Technical Metrics

- **Processing Accuracy:** > 95% correct transaction extraction
- **System Uptime:** > 99.5% availability
- **Performance:** < 500ms average API response time
- **Error Rate:** < 1% processing failures

12.2 User Experience Metrics

- **User Adoption:** 70% of users actively use the system after 30 days
- **Time Savings:** 90% reduction in manual data entry time
- **User Satisfaction:** > 4.5/5 star rating from users
- **Feature Utilization:** 80% of core features used by active users

12.3 Business Metrics

- **Cost per Transaction:** < \$0.01 per processed transaction
- **Data Coverage:** Support for 80% of major Indian banks
- **Processing Volume:** 10,000+ transactions processed monthly
- **User Growth:** 50% month-over-month growth in active users

Appendices

Appendix A: Bank Statement Format Analysis

- Detailed format specifications for each supported bank
- Regular expressions for transaction extraction
- Edge cases and handling strategies

Appendix B: API Documentation

- Complete API specification with request/response examples
- Authentication flow documentation
- Error code reference

Appendix C: Deployment Guide

- Step-by-step deployment instructions
- Environment configuration
- Troubleshooting guide

Appendix D: Contributing Guidelines

- Code style and conventions
- Pull request process
- Development setup instructions

Document Approval:

- ☐ Technical Lead Review
- ☐ Product Owner Approval
- ☐ Security Team Review
- ☐ Architecture Committee Approval

Next Review Date: September 15, 2025